

Enterprise AI agent security

▲ ● ■ Школа анализа данных

How to Use Agents to Speed Up Work
and Not Lose Data, Control, or Infrastructure

Andrey Romanovsky
Security engineer teamlead



Contents

- 01 Why both bans and full freedom fail
- 02 What a corporate AI agent is made of
- 03 How to properly share company data with an AI agent
- 04 How to control agent access inside and outside the company
- 05 How to control an agent's tools and actions
- 06 How to set up monitoring and run agents securely

AI has already entered our workplace

Development code generation code review migrations tests documentation	Analytics and office tasks document summarization knowledge base search preparing reports processing spreadsheet	Infrastructure and DevOps log analysis config generation pipeline execution managing incidents
Information security analyzing vulnerabilities alert triage assisting with investigations detection rule generation	Design content preparation photo/video generation drafting design documents	And much more Almost any field

Neither extreme works

Ban everything

Pros:

- formally less risk
- easier to explain the policy
- fewer integrations

Cons:

- users find workarounds
- Shadow AI emerges
- data leaks into personal accounts and external chats
- the company loses observability
- decrease in productivity

Allow everything

Pros:

- maximum productivity
- access to top AI models
- less friction for users

Cons:

- leaks of personal data, NDAs, code, secrets
- unclear who sent what and when
- external providers become part of the data processing chain
- agents can acquire excessive privileges

There is a third way — controlled usage

Security must not come at the cost of usefulness

Agent security = controlling autonomy,
not banning capabilities

What we want to control:

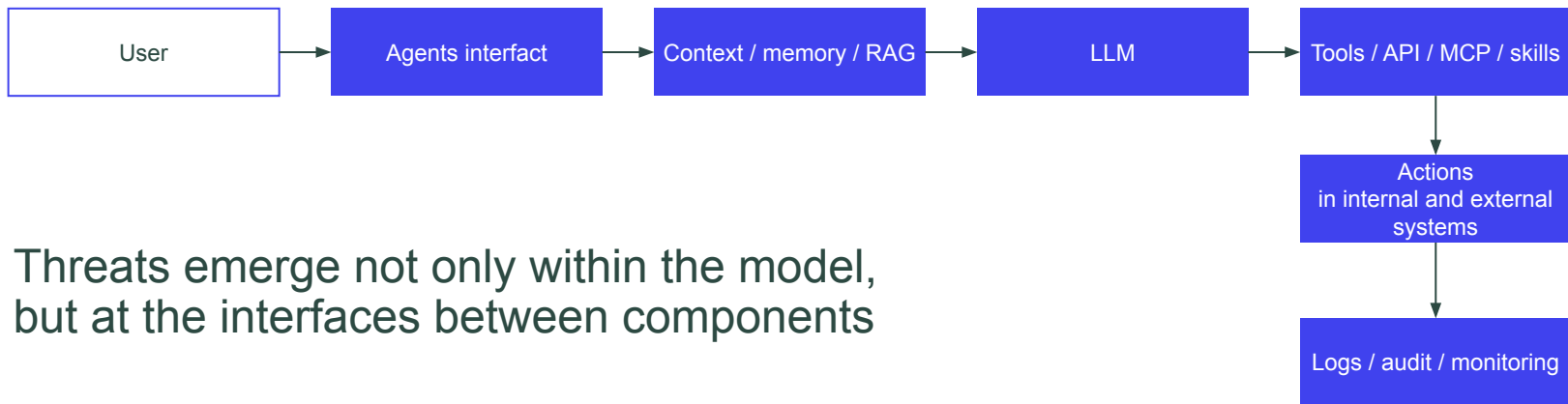
- what data the agent can see
- what models it can use
- what tools are available
- what actions can be taken automatically
- what actions require approval
- where code is executed
- what is logged and monitored

What we don't want to lose:

- development speed
- user experience
- model quality
- automation of routine tasks
- the ability to experiment

An agent is not a chat

It is a system with permissions and actions



Threats emerge not only within the model,
but at the interfaces between components

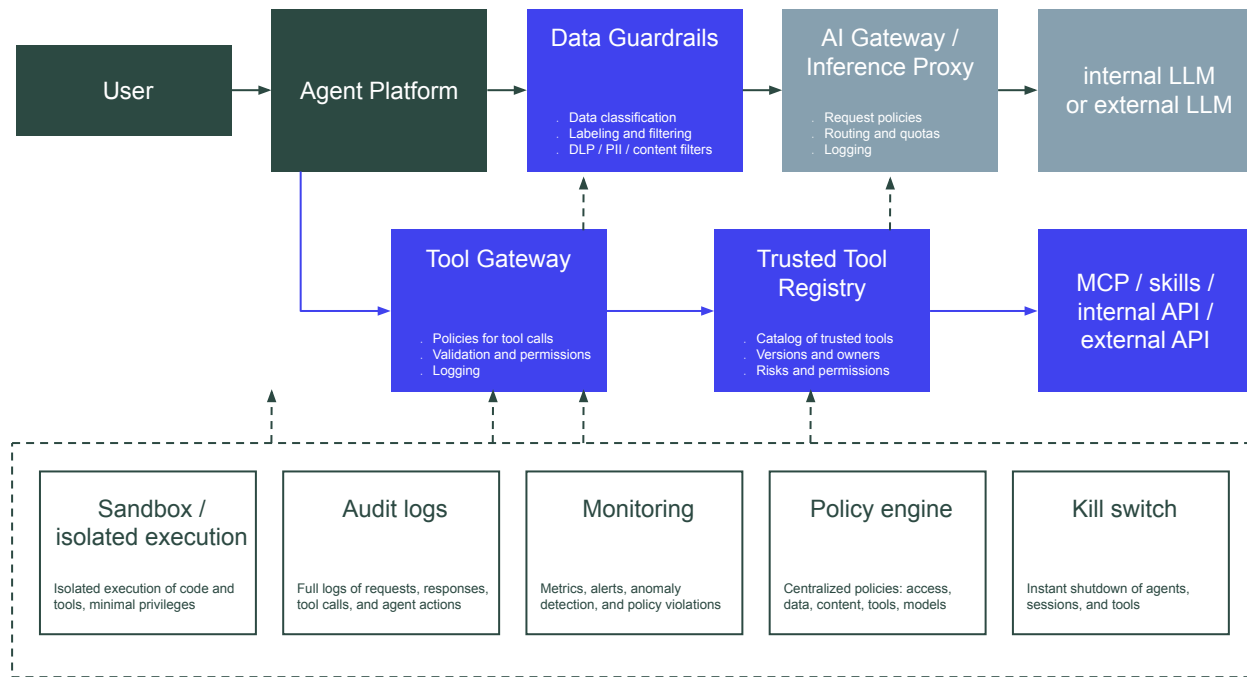
Highlights:

- data enters the context
- the model decides what actions to take
- tools modify the state of systems
- logs serve as the basis for investigations
- the agent's permissions determine the scope of damage

Where the security of a corporate agent fails

1. Input data PII, NDAs, secrets, excess context, malicious documents	2. Model and interface internal or external model, provider, logging, training on data	3. Context and memory long-lived data, task mixing, cross-session leaks
4. Instruments MCP, skills, APIs, external services, malicious or excessively broad tools	5. Agent actions write operations, deletion, permission changes, sending data outside	6. Execution environment localhost, sandbox, network, file system, secrets

Basic architecture for secure use of agents



The agent must not access models, tools, or external services directly!

On-premise, external, and combined inference

On-premise inference

Pros:

- maximum control
- data stays within the perimeter
- easier to meet sensitive data requirements

Cons:

- models may be weaker
- more expensive to maintain
- less model variety
- access control inside the company is still required

External inference

Pros:

- access to the best and most up-to-date models
- fast access to new capabilities
- no need to maintain your own infrastructure

Cons:

- data leaves the perimeter
- vendor dependency
- hard to guarantee how data is handled downstream
- contracts, policies, and technical filters are required

Combined approach

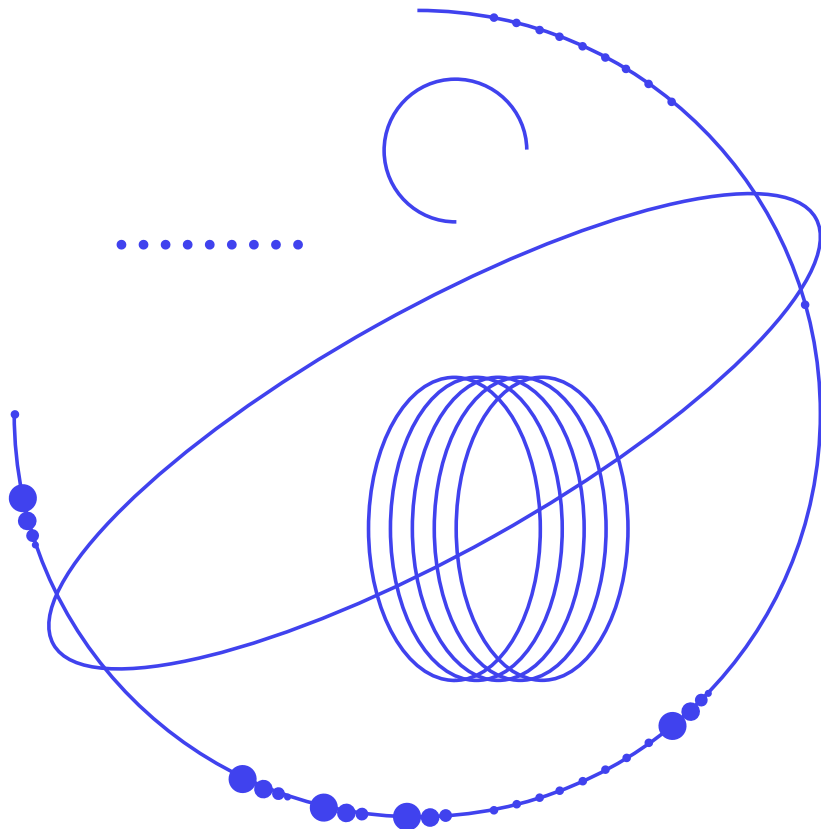
Pros:

- sensitive data — routed to internal models
- non-sensitive data — routed to external models
- balance between quality and control

Минусы:

- data routing is required
- guardrails are needed
- auditing is required
- a classification error can result in a data leak

Data for a corporate AI agent



Not all data is the same for AI

D0. Public

Examples: public documentation, open-source code, publicly available articles

Permissions: can be sent to external models

D1. Internal

Examples: internal guidelines, company-wide documents without sensitive content

Permission: can be used with corporate AI tools; external inference — subject to policy

D2. Confidential / NDA

Examples: internal plans, architecture, unreleased products, confidential research

Permissions: internal inference only or a specifically approved secure environment

D3. Regulated / Sensitive

Examples: personal data, financial records, commercial secrets, infrastructure security information

Permissions: internal inference only, data minimization, masking, strict auditing

D4. Secrets

Examples: tokens, keys, passwords, private certificates

Permissions: must never reach the model; use references, identifiers, or a secrets vault only

First quality gate — don't pass unnecessary data (Data Minimization)

Bad pattern:

"Let's send the model the full ticket, the entire thread, all logs, the whole file, and all context — it'll figure it out."

Good pattern:

"We'll extract only the fields needed for the task: error description, stack trace without tokens, relevant log lines, anonymized IDs, minimal code snippet."

Data minimization techniques:

- allowlist-based field selection
- context size limitations
- removal of irrelevant content
- access-aware RAG
- summarization before sending
- replacing sensitive values with placeholders
- block full attachment passthrough without analysis

Data sanitization should not happen in just one place

5 possible control points:

1. **Before data enters the agent's context:**

Filter user input, attachments, and pasted text

2. **When retrieving data from RAG / internal systems:**

Check permissions, resource classification, and field sensitivity

3. **Before sending to the model:**

PII/NDA redaction, secrets detection, policy-based routing

4. **Before calling an external tool:**

Verify that no sensitive context is being sent outside

5. **After the model's response:**

Output guardrail: check that the model hasn't returned PII, secrets, NDAs, or a dangerous payload

Guardrails are not a single filter —
they are a set of checks at the boundaries of the system

PII guardrails: what needs to be detected

Categories

Direct identifiers

- Full name
- Email
- Phone
- Passport details
- Address
- Employee or customer IDs

Financial and contractual data

- Card and account numbers
- Salaries
- Contracts and amounts

Technical identifiers that may be sensitive

- Cookie / auth headers
- Device ID
- IP addresses

Secrets

- API keys
- OAuth tokens
- private keys
- Kubeconfig
- Cloud credentials
- Passwords
- Recovery codes

Quasi-identifiers

- combinations of fields that are safe individually but together reveal a person or entity

NDA's can't be detected with regex alone

Examples of NDA-protected data:

- unpublished code
- architectural diagrams
- internal release plans
- names of confidential projects
- research findings
- internal incidents
- customer and partner data
- non-public financial metrics
- internal security findings
- vulnerabilities prior to remediation

Why this is difficult:

- NDA status is often context-dependent
- the same text can be public or confidential
- there's no universal regex for "secret roadmap"
- data sources and owners must be labeled
- what matters is not just the content, but where the data came from

Sanitization pipeline: from raw context to a safe prompt

Pipeline diagram

1. **Collect**

Gather the user query, RAG fragments, and tool outputs

2. **Classify**

Determine data classes: public / internal / confidential / PII / secrets

3. **Decide**

Choose an action: allow, sanitize, route to an internal model, or block

4. **Transform**

Mask, remove, summarize, or replace with placeholders

5. **Route**

Select the model: internal, external, specialized, or reject the request

6. **Log safely**

Log the decision and metadata, but do not store the full sensitive text

Transformation example

Before: "Ivan Petrov,
ivan.petrov@example.com, salary
350000, error in service X"

After: "[EMPLOYEE_1], [EMAIL_1],
[SALARY_REDACTED], error in
[INTERNAL_SERVICE_1]"

Sanitization is not just about masking with asterisks

4 approaches

1. **Removal**

Delete the field entirely. Example: Authorization: Bearer ... → [REMOVED_SECRET]. Use for secrets and explicitly prohibited data

2. **Masking**

Preserve part of the value. Example: +7 999 123-45-67 → +7 *** ***-**-67. Use when the format matters but the value does not

3. **Pseudonymization**

Replace with a stable placeholder. Example: Ivan Petrov → [PERSON_1]. Use when relationships within the text must be preserved

4. **Generalization / summarization**

Replace specifics with meaning. Example: "salary 350,000" → "highly sensitive financial value". Use for NDA content and financial details

The choice of method depends on what the model needs to do

Do we need to be able to recover data after cleanup?

Two modes

Irreversible redaction

Data is permanently removed with no possibility of recovery.

Suitable for:

- secrets
- data not needed to generate a response
- external models
- low-trust scenarios

Reversible pseudonymization

Data is replaced with placeholders, and the mapping is stored within a secured perimeter.

Suitable for:

- internal models
- scenarios where the user needs a correct, complete result
- document processing where relationships between entities must be preserved

Transformation example

Prompt sent to the model:

"[PERSON_1] confirmed a meeting with [PERSON_2]"

Post-processing within the company:

"Ivan confirmed a meeting with Maria"

A guardrail decision is not just block or allow

Possible solutions

- Allow → The request is safe; proceed with the selected model
- Mask and allow → PII or internal identifiers detected; the request can proceed after sanitization
- Route internal → Data is sensitive but the scenario is legitimate; send to the internal model only
- Ask user to reduce context → Too much unnecessary data; ask the user to remove the file, secrets, or irrelevant fields
- Human review → Ambiguous scenario: possible NDA content or a high-risk action
- Block → Secrets, prohibited data, attempted exfiltration, or a dangerous tool combination

The right place to enforce NDA controls is at the source

Examples of resources:

- knowledge base page
- ticket
- repository
- folder with documents
- log storage
- table
- CI/CD artefact
- security report

Resource attributes:

- Permission to load into AI agent — `ai_allowed=true/false`
- Permission to send to external models — `external_ai_allowed=true/false`
- Contains PII — `contains_pii=true/false/unknown`
- Contains secrets — `contains_secrets=true/false/unknown`
- Data class — `data_class=public/internal/confidential/restricted`
- Data owner — `owner=team/service`
- Client-side data retention — `ttl/cache_allowed=true/false`

The model should not be the only DLP

Anti-pattern:

«Before sending data externally, we'll ask the LLM to remove all PII and secrets»

Why this is bad:

- the model may miss data
- the model may hallucinate classifications
- a prompt injection attack can instruct it not to remove secrets
- there is no determinism
- correctness is hard to prove
- regression testing is difficult

Best practice:

- deterministic checks for simple entities
- ML/NER classifiers for PII
- secret scanners
- DLP/policies for NDA content
- LLM-as-a-judge only for complex and ambiguous cases
- the final decision is made by the policy layer, not the LLM itself

Logs can also become a leakage channel

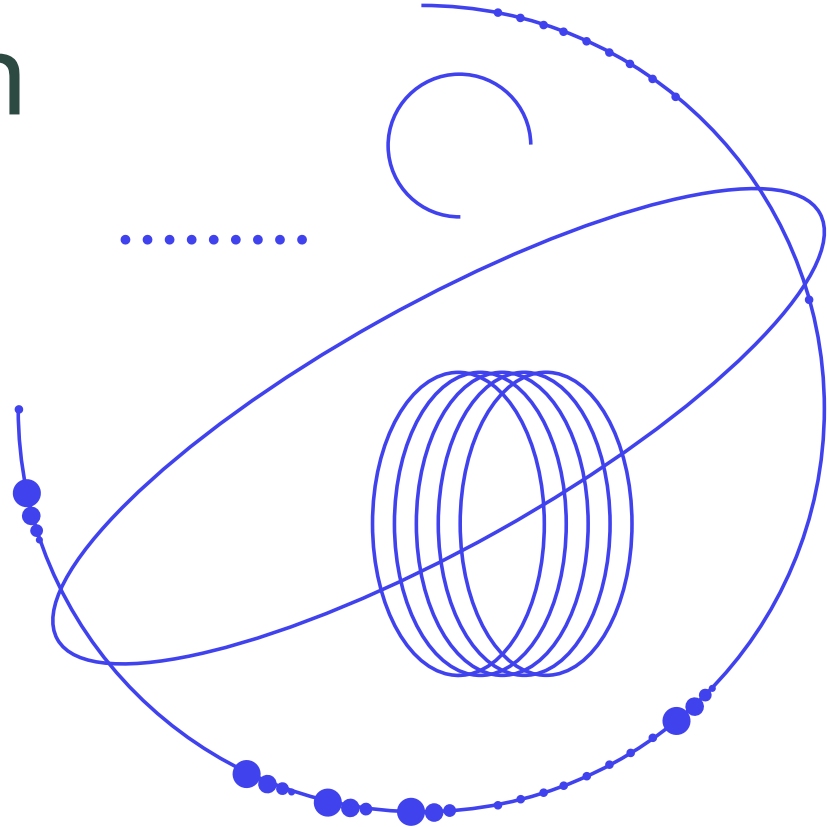
What is dangerous to log:

- the full prompt
- the full model response
- attachments
- RAG documents
- secret values
- PII before masking
- the placeholder-to-real-value mapping table
- tool call payloads without filtering

What is safer to log:

- trace ID
- user ID / agent ID
- data class
- guardrail decision
- selected route
- model / provider
- tool name
- outcome
- whether data was redacted
- hashes or truncated previews
- references to a secured store with TTL

Access controls for an AI agent within the company



An agent needs its own identity!

Why acting solely "on behalf of the user" doesn't work:

- logs don't show where the human ends and the agent begins
- incidents are difficult to investigate
- the agent inherits too many permissions
- agent behavior cannot be restricted independently
- rate limits and budgets cannot be managed separately

The right model:

- a dedicated identity for the agent
- linked to the owning user or team
- minimal permissions
- separate secrets
- separate limits and quotas
- separate action logs
- the ability to quickly disable the agent

A single broad token is an anti-pattern! (Agent Least Privilege)

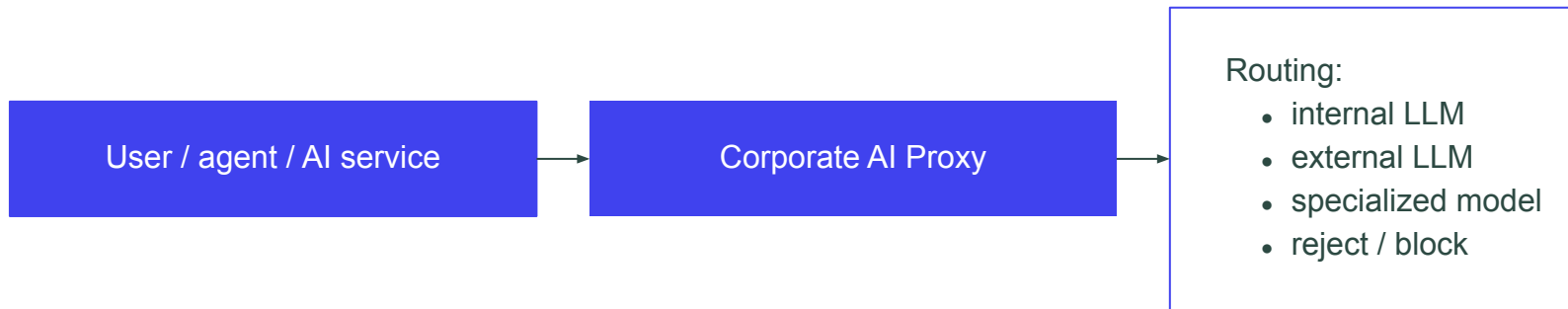
Anti-pattern:

- one token for all operations
- permissions matching a user or service account
- long-lived secret
- read and write access at the same time
- no task-level restrictions
- no separate audit trail

Best practice:

- scoped token
- short TTL
- specific operation
- specific resource
- specific agent
- specific initiating user
- separate approval for write actions
- token revoked after execution

A single egress point for model access (AI proxy and model infrastructure)



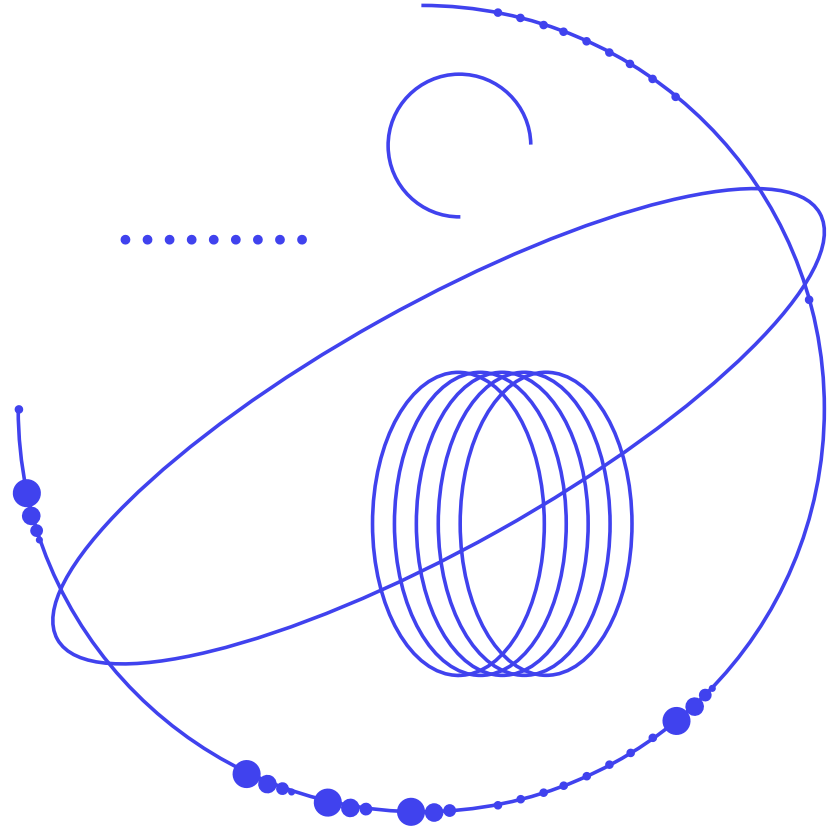
Why an AI proxy is needed

- Authorization and identification
- Data classification and sanitization
- Request routing
- Policies and rate limits
- Logging and auditing
- Emergency controls

What to monitor in internal models

- Pre-runtime access controls
- Logs
- Secure deployment
- Rate limits

AI agent tools and actions within the company



Tools must be approved through the trusted registry (Secure Tool Registry)

What the tool registry stores

- owner
- version
- description
- endpoint
- type: read-only / write / external / internal
- required permissions
- allowed models
- allowed data classes
- security review results
- date of last review
- list of known risks
- update policy

Forbidden pattern

The agent discovers an MCP server on its own, reads its description, and starts making calls

Correct pattern

The agent only sees tools from a trusted registry via the Tool Gateway

A tool call is a security boundary (Secure Tool Gateway)

What the Tool Gateway checks

Before the call:

- who the agent is
- who the initiating user is
- which tool is being called
- read or write
- what the parameters are
- whether sensitive data is present
- whether the tool combination is permitted
- whether human approval is required

Во время вызова

- rate limits
- timeout
- network allowlist
- sandbox
- resource limits

After the call:

- result filtering
- PII/NDA redaction
- checking the tool response for prompt injection
- logging
- deciding whether the result can be returned to the agent's context

Tools that are safe individually can be dangerous in combination

Example 1

Read internal docs + Send external message→ an exfiltration channel

Example 2

Read logs + Create public issue→ leakage of PII or secrets

Example 3

Read vulnerabilities + Send to external model→ leakage of security findings

Example 4

Create config + Deploy→ uncontrolled changes to the production environment

Mitigations

- policies on tool combinations
- prohibit external tools when sensitive context is in scope
- taint tracking for data
- approval required before sending data outside
- separate agents for different risk zones

Read and write carry different levels of risk

Read-only actions:

- search
- reading a document
- log analysis
- fetching a status
- generating a summary

Controls:

- access rights check
- data filtering
- auditing
- rate limits

Write actions:

- delete
- modify a config
- grant permissions
- send an email
- create a task
- run a pipeline
- deploy

Controls:

- plan before execute
- dry-run
- human approval
- rollback
- extended logging
- deny execution under uncertainty

When human involvement is required

Critical action: an action is classified as critical when an error could result in damage to the company, users, customers, or infrastructure

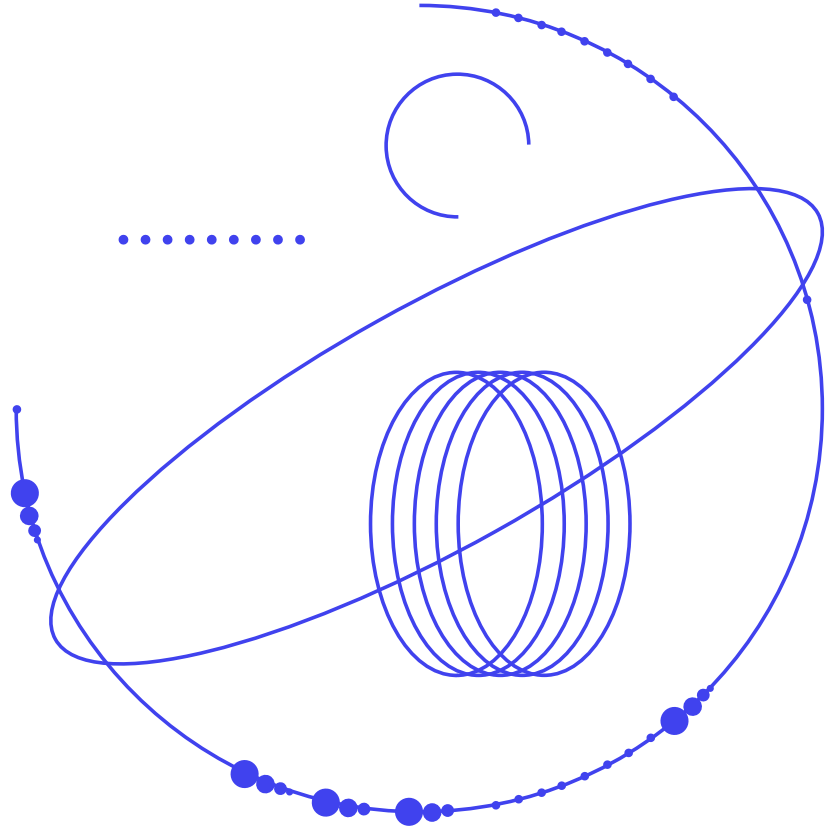
Examples of critical actions:

- deleting data
- modifying access permissions
- changing production configuration
- deploying
- sending an external message containing internal data
- processing personal data
- financial transactions
- publishing on behalf of the company
- changing security settings

When approval is not required:

- reading permitted data
- creating a draft
- local analysis
- dry-run
- generating a recommendation
- actions within a sandbox

How to monitor and securely run an AI agent inside the company



If an agent executes code, it needs
a sandbox

When an incident happens, you need facts — not assumptions

What to log

- agent_id
- user_id / initiator
- task_id / trace_id
- model used
- external or internal inference
- data class
- redaction result
- tools invoked
- tool call parameters after sanitization
- approval status
- action outcome
- guardrail violations
- policies triggered

What we don't log in plain text

- full PII
- secrets
- NDA documents
- original attachments
- de-pseudonymization tables
- full prompts unless strictly necessary

The goal is investigation, reproducibility, cost control, and anomaly detection

You need to be able to stop an agent

Monitoring signals:

- sharp spike in token usage
- increase in external requests
- surge in guardrail blocks
- attempts to send PII externally
- unusual tool calls
- new tool combinations
- recurring errors
- high volume of write actions in a short time
- suspicious commands in the sandbox
- attempts to access restricted files

Kill switch levels

- disable a specific agent
- disable a specific tool
- block all write actions
- disable external inference
- switch the agent to read-only mode
- shut down the entire platform

An agent should be deployed like a product, not launched like a prompt

Before launch, you need to specify

- Agent purpose
- Owner
- Risk classification
- Data used
- Models used
- Tools used
- Read/write capabilities
- Where approval is required
- Sandbox boundaries
- Which guardrails are enabled
- What is logged
- How to disable the agent
- How security is tested
- Who responds to incidents

If this is not defined, the agent
is not ready for production use

Requirements should be risk-based: a classification example

R0. Sandbox / experiment

- test data only
- no access to production
- no write actions
- minimal requirements

R2. Operational agent

- works with internal systems
- can trigger processes
- limited write actions permitted
- approval, sandbox, and extended audit required

R1. Assistant

- reads non-critical data
- creates drafts
- does not perform critical actions
- basic guardrails and audit logging

R3. Critical agent

- production, finance, PII, access management, security
- strict review required
- mandatory human-in-the-loop
- isolation
- continuous monitoring
- regular red teaming

Minimum security checklist for a corporate AI agent

- The agent has an owner and a defined purpose
- A risk class has been assigned
- The agent has a dedicated identity
- Permissions follow the least privilege principle
- Data in use is documented
- Data is classified and minimized
- PII, NDA content, and secrets are sanitized before reaching the model
- External inference goes only through a proxy or gateway
- Tools are accessible only through a trusted registry or gateway
- Write actions require approval
- Code runs in a sandbox
- Logs are safe and sufficient for incident investigation
- Monitoring and a kill switch are in place
- Security regression tests exist
- There is a review process when new tools, data, or models are added

A secure agent is not just a "good prompt"

Key takeaway: AI agent security within a company is built not around a single model, but around a system of controls

The system of controls:

- data classification
- context minimization
- PII / NDA / secrets sanitization
- controlled inference
- agent identity
- least privilege
- trusted tools
- tool gateway
- sandbox
- human approval
- audit
- monitoring
- kill switch

AI can be used safely if you design not just the agent, but the boundaries of its autonomy

Thank you for your attention

Andrey Romanovsky
Security engineer teamlead

